



Vector Store & Feature Store

AICB-P2T2 · Ngày 19 · Chương 4: Hạ Tầng

Giảng viên

VinUniversity · Phase 2 · Track 2 · Tuần 4

HÃY SUY NGHĨ...

“SQL database trả về exact match. Nhưng AI cần “tương tự” — semantic search. Tại sao SQL không đủ cho AI search, và vector database thực sự lưu gì?”

Giữ câu hỏi này trong đầu khi học bài hôm nay

1. Vector Embeddings: Text $\rightarrow$ Số
2. Vector DB Landscape
3. ANN & Retrieval Patterns (HNSW, Hybrid)
4. RAG Pipeline với Vector Store
5. GraphRAG & Knowledge Graphs
6. Feast Feature Store
7. Production Patterns + Case Studies
8. Demo: Semantic Search API

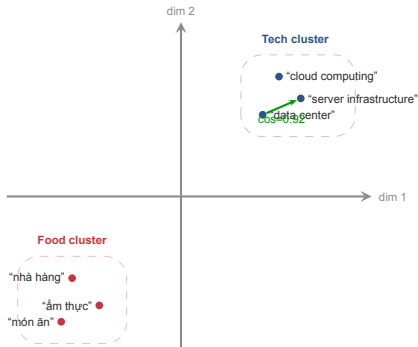
Sau buổi học này, bạn sẽ:

1. Hiểu vector embeddings và cách ánh xạ text thành dense vectors
2. Deploy và query vector database (Qdrant/Weaviate/Pinecone)
3. Xây dựng RAG pipeline hoàn chỉnh với hybrid search
4. Setup Feast Feature Store với offline/online store

Embeddings → Vector DB → ANN + Hybrid → RAG → GraphRAG → Feature Store
→ Production

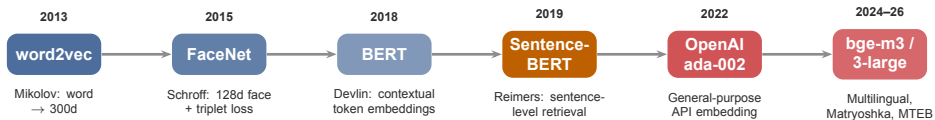
Vector search endpoint với semantic similarity + Feature Store offline/online

- REST API `/search?q=...` trả về top-10 semantic results < 50 ms
- Hybrid search: vector + BM25 keyword, merge với RRF
- Feast: 3 feature views, materialize, online lookup test
- Benchmark: keyword search vs semantic vs hybrid



- text-embedding-3-small: OpenAI, 1536d
- text-embedding-3-large: 3072d, Matryoshka
- bge-m3: BAAI, 1024d, multilingual
- nomic-embed: open-source
- PhoBERT: VN lightweight baseline (xem bảng 2026)

Cosine Similarity — $\frac{\vec{A} \cdot \vec{B}}{|\vec{A}| \times |\vec{B}|}$ — giá trị 0–1, >0.85 = very similar



2013–2019: task-specific embeddings — mỗi domain (NLP, face, audio) train riêng. **2020–2026:** general-purpose foundation embeddings — một model, nhiều use cases, multilingual, multi-modal. Re-index cost cao ⇒ chọn cẩn thận từ đầu.

Model	Dim	ML	MTEB	Cost
text-embedding-3-small	1536	✓	62.3	\$0.02/M
text-embedding-3-large	3072	✓	64.6	\$0.13/M
bge-m3	1024	✓	59.5	self-host
multilingual-e5-large	1024	✓	~64	self-host
nomic-embed-text-v1.5	768	EN	62.4	self-host
PhoBERT	768	VN-only	baseline	self-host

Prod tiếng Việt 2026: bge-m3 (self-host) hoặc

text-embedding-3-large (API). PhoBERT chỉ cho on-device nhẹ.

- text-embedding-3-large dùng MRL — truncatable: 3072d → 256d
- **256d vẫn thắng ada-002 @ 1536d** trên MTEB
- Tiết kiệm RAM/disk 6×, giảm chi phí vector DB
- Áp dụng ngay khi scale > 10M vectors

Lưu ý: Đổi embedding model = re-index toàn bộ. Train và serve **phải** dùng cùng model version — xem §6 blue-green lifecycle.

$$\cos(\vec{a}, \vec{b}) = (\vec{a} \cdot \vec{b}) / (\|\vec{a}\| \|\vec{b}\|)$$

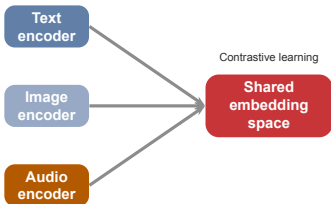
- Range $[-1, 1]$, angle only
- Default cho text embedding
- OpenAI/BGE/e5 unit-norm $\Rightarrow$ dot = cosine
- Match metric với pretraining objective: sai metric degrade recall 10–20%

$$\vec{a} \cdot \vec{b} = \sum a_i b_i$$

- Bao gồm magnitude
- Nhanh hơn (no division)
- ColBERT, DPR dùng dot

$$d = \sqrt{\sum (a_i - b_i)^2}$$

- Triangle inequality $\Rightarrow$ IVF / kd-tree
- CLIP image, wav2vec audio
- Unit-norm:
 $d^2 = 2(1 - \cos)$



CLIP (OpenAI): text+image, foundational

SigLIP (Google 2024): better zero-shot

jina-clip-v2: multilingual incl. tiếng Việt

voyage-multimodal-3: text+image+table

ImageBind (Meta): 6 modalities

E-commerce visual search: chụp ảnh → tìm sản phẩm (Tiki, Shopee). **Content moderation**: ảnh+caption matching policy. **Video indexing**: TikTok/YouTube tìm clip theo text prompt. **Medical**: X-ray+report đồng embedding cho clinical search.

Feature	Pinecone	Weaviate	Qdrant	pgvector
Hosting	Managed	Self/Cloud	Self/Cloud	Extension
API	REST/gRPC	GraphQL	REST/gRPC	SQL
Multi-modal	×	✓	×	×
Filtering	Metadata	GraphQL	Payload	SQL WHERE
P99 (1M vecs)	8–22 ms	~10 ms	~5 ms	15 ms
Quantization	✓	✓	✓ (binary)	scalar
Best for	Production	Multi-modal	Self-hosted	Have Postgres

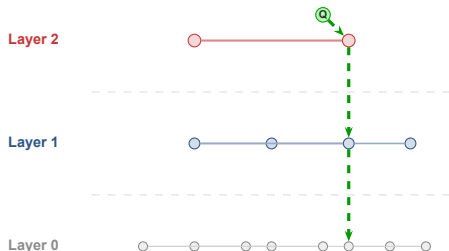
Nguồn:

VectorDBBench 2025 · 1M vectors, 768 dims, HNSW index, top-10 query. pgvector+pgvectorscale (Timescale DiskANN).

- Library, *not* a database — C++/Python, no server
- GPU: 1B vectors, top-K < 1 s
- IVF, HNSW, PQ, OPQ, IVF-PQ — cùng một API
- In-memory + on-disk indices
- 2026: vẫn là embedded tier bên trong Milvus, Vespa, pgvector-rs

- **Face recognition:** FaceNet, ArcFace — FaceID, photo search
- **Pinterest visual lens:** CNN feats + FAISS, 2017+
- **RecSys candidate gen:** YouTube/Spotify/TikTok — ~1B vectors
- **Plagiarism / dedup:** Turnitin, news article clustering
- **Audio fingerprinting:** Shazam, Spotify duplicate detection

Deep Metric Learning — Pre-LLM vector retrieval dựa vào triplet loss (FaceNet), contrastive loss, ArcFace angular margin — train embedding sao cho *cosine* (hoặc Euclidean) tách class. Đây là tiền thân của contrastive pretraining hiện đại (CLIP, SigLIP).



- Graph-based, multi-layer skip list
- Recall 95%+ ở ~ 10 ms (in-memory)
- Params: $ef=200$, $m=16$ (default tốt)
- Native: Pinecone, Weaviate, Qdrant, Milvus

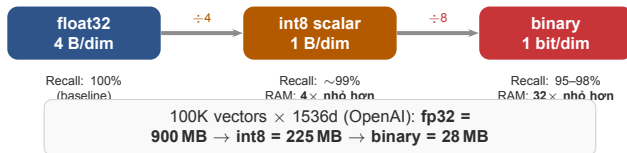
- RAM-bound: $>10M$ vec @ 768d $\rightarrow$ tốn ~ 10 GB
- Update-heavy corpus: re-build chậm
- Static, billion-scale $\rightarrow$ DiskANN (slide tiếp)

- Graph in-memory, multi-layer skip list
- Recall 95%+ @ ~ 10 ms
- RAM-bound: ~ 1 KB/vec (768d fp32)
- **Best for:** ≤ 10 M hot vectors, latency-critical
- Params: ef=200, m=16

- Cluster-based inverted index
- Lower RAM, higher latency (~ 30 – 50 ms)
- Retraining cần khi data drift lớn
- **Best for:** batch-mode, static corpus, CPU-only infra
- Params: nlist=1024, nprobe=64

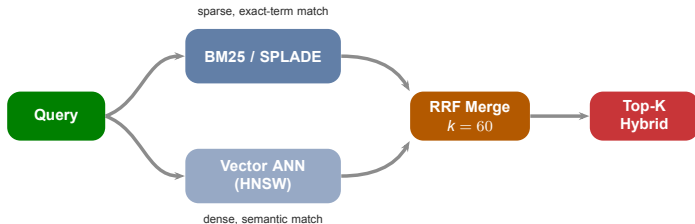
- Graph trên SSD — billion-scale 1 node
- Latency ~ 30 ms @ 99% recall
- **10–50 $\times$ rẻ hơn HNSW** ở scale lớn
- Dùng bởi: Pinecone serverless, pgvectorscale, Azure Cosmos DB
- **Best for:** > 100 M vectors, cost-sensitive

RaBitQ (SIGMOD 2024 Best Paper) — Unbiased binary quantization cho DiskANN — tích hợp trong pgvectorscale & VectorChord. Giảm RAM thêm $32\times$ với chất lượng recall tương đương.



- **Binary stored (28 MB) + scalar query** (precise)
- Best of both: cheap storage + high recall
- Recall $\geq 99\%$ với chi phí binary storage

- Qdrant, Weaviate, Pinecone, Milvus: hỗ trợ binary/scalar out-of-the-box
- Bật quantization **trước khi ingest** — không phải sau
- Re-quantize = re-index $\Rightarrow$ blue-green (xem §6)

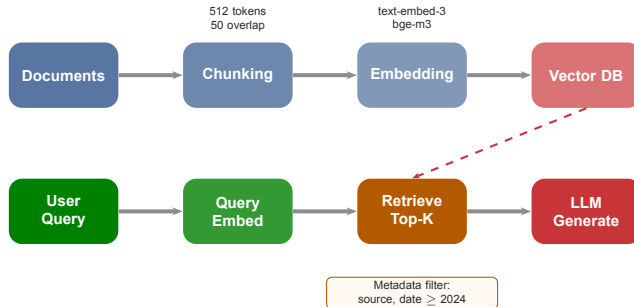


Reciprocal Rank Fusion — $\text{score}(d) =$

$$\sum_r \frac{1}{k + \text{rank}_r(d)} \quad (k = 60)$$

Rank-only: không cần normalize raw scores giữa BM25 (TF-IDF) và cosine.

- Recall@10: **91% hybrid** vs 78% dense-only
- Latency overhead: **+6 ms** (parallel retrieval)
- Storage: **1.4×** (BM25 inverted index + vector)
- Native: Qdrant, Weaviate, OpenSearch, Elasticsearch, Milvus
- **SPLADE** (sparse GPU): recall > BM25, nhưng cần GPU inference — chọn BM25 cho cost-sensitive



- **Fixed-size:** 512 tokens, baseline, ngắt giữa câu
- **Recursive:** LangChain RecursiveCharacterTextSplitter — tôn trọng câu/đoạn
- **Semantic:** similarity-based, nghỉ tại topic shift — chậm + tốt nhất
- **Hierarchical:** parent (page) + child (paragraph), retrieve child, return parent

- **Chunk size:** 200–500 tokens cho RAG, 1K+ cho long-context
- **Overlap:** 10–20% (~50–100 tokens)
- **Tooling:** LangChain, LlamaIndex, Unstructured.io
- **Vietnamese:** tokenize ở sentence level, không byte-split

Lưu ý: Đo bằng **golden set**: Recall@k cải thiện đáng kể khi switch fixed → semantic trên benchmarks. Bad chunking = embedding tốt cũng vô dụng.

- **Top-K:** 5–10 cho RAG, 20–50 trước reranker
- **ef_search** (HNSW): 200 default, tăng cho recall
- **MMR diversity:** $\lambda=0.5$ giảm chunks trùng
- **Metadata filter:** apply sớm, giảm search space

- Embedding model consistency: train = inference
- Query rewriting: HyDE, multi-query expansion
- Embedding versioning + zero-downtime re-index (xem §6)
- Monitor embedding drift hàng tuần

Lưu ý: Training và inference **phải** dùng cùng embedding model version. Đổi model = re-index toàn bộ.

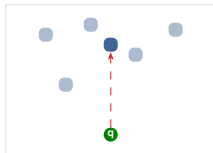
- ANN search trả về top-100 candidates
- Bi-encoder: query & doc embed **riêng**
- Latency thấp (<20 ms), recall cao

- ms-marco-MiniLM-L-6-v2: nhẹ, nhanh
- bge-reranker-v2-m3: multilingual, chính xác
- Cohere Rerank API: managed service

- Cross-encoder: query + doc **cùng lúc**
- Score relevance từng cặp (query, doc)
- Chọn top-10 chất lượng cao nhất

- **Self-host:** sentence-transformers trên GPU — kiểm soát, tiết kiệm ở scale lớn
- **API:** OpenAI text-embedding-3-small, Cohere embed v4 + Rerank 3.5 — đơn giản, triển khai nhanh

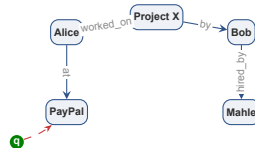
Vector RAG (chunks)



Top-K nearest chunks
(cosine similarity)

“Ai ở PayPal từng cộng tác với người được Mahle thuê?” — Vector trả về chunks về **PayPal** HOẶC **Mahle** riêng lẻ; không thể **cross-document multi-hop**. KG traverse 3-hop trong μs .

GraphRAG (entity-relation)



Multi-hop: PayPal → Alice →
Project X → Bob → Mahle

- **P3C diabetes copilot** (Memgraph): patient journey + drug interactions
- **Alzheimer research**: 1.6M edges nối genes-drugs-trials
- **M&A intel** (GlassDollar/Siemens, Mahle): entity search across millions of companies

F

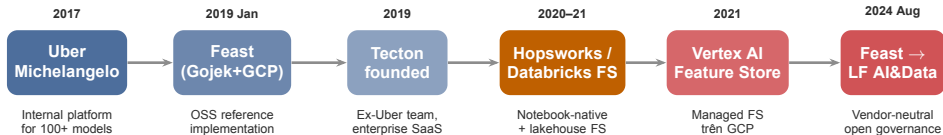
Vector RAG = “đoạn văn liên quan”. GraphRAG = “mối quan hệ kết nối nhiều entity” — chọn graph khi câu trả lời là một **relationship**, không phải đoạn text.



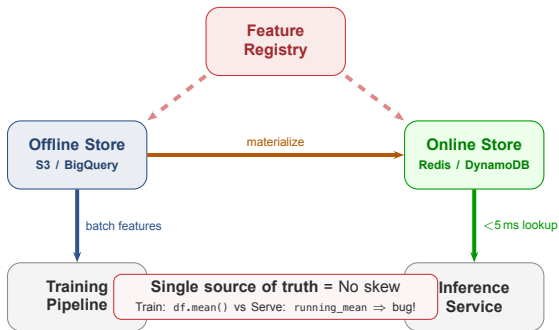
- **Microsoft GraphRAG:** end-to-end + community summaries (LLM-heavy, cost cao)
- **LightRAG:** dual-level retrieval, graph-enhanced indexing, rẻ hơn MS GraphRAG
- **Neo4j** + LangChain: managed graph DB + Cypher query
- **Memgraph:** in-memory graph, sub-ms traversal (P3C copilot)

1. **Start:** vector RAG cho conversational grounding
2. **Add KG:** cho entity-rich domain (legal, medical, M&A)
3. **Hybrid:** vector retrieve → KG expand entities → LLM
4. **Cost:** KG construction = 5–20× vector indexing

Lưu ý: Cost reality: LLM-driven NER + relation extraction cho 100K docs ~\$5K–\$20K (GPT-4o). Bắt đầu nhỏ: 1 domain, 1K docs, validate retrieval lift trước khi scale.



Uber có 100+ ML models reuse cùng features (rider price, driver ETA) ⇒ centralize để tránh skew + duplicate compute. **LLM-era twist:** feature stores giờ host cả *embedding feature views* (user/item vectors) bên cạnh tabular features — một hệ thống cho cả ML cổ điển và RAG personalization.



- `FeatureView(name="user_features")`
- Entities: ["user_id"]
- TTL: `timedelta(days=30)`
- Source: Delta/Iceberg tables (N18)

- `store.get_online_features()`
- Latency: <5 ms per request
- Batch: 1000 entity rows at once

- `feast materialize-incremental`
- Batch features: daily update là đủ
- Streaming features: cần sub-second
- Feast 0.40+ (LF AI&Data): Push API GA, on-demand transformations Beta, Prometheus metrics built-in

- **Tecton**: fully managed, real-time features
- **Databricks**: Feature Engineering in Unity Catalog (Workspace FS đã legacy)
- **Vertex AI FS**: GCP managed service
- **Hopsworks**: regulated industries, on-prem

- Parquet trên S3 / Delta / Iceberg
- Snowflake / BigQuery / Redshift
- **Workload:** historical batch, 100GB–TB
- **Latency:** seconds–minutes (OK for training)
- Lưu *full history* của mọi feature value

- Redis / DynamoDB / Cassandra / Aerospike
- **Workload:** per-entity KV lookup
- **Latency:** <10 ms P99
- Lưu *current value only* (no history)
- Feast `materialize()` nạp offline → online

Point-in-Time (PIT) Join — Khi build training set: lấy feature *as-of timestamp* của mỗi event — không dùng giá trị tương lai. Feast `get_historical_features(entity_df)` thực hiện PIT join trên offline store. **Sai lầm:** dùng LATEST value $\Rightarrow$ data leakage $\Rightarrow$ prod accuracy thấp hơn training 20–30%.

- LF AI&Data, cộng đồng mạnh
- Push API, on-demand transformations (Beta)
- Self-host: nhẹ, kiểm soát hoàn toàn
- **Best for:** team nhỏ-vừa, full control

- Real-time features, sub-second freshness
- DAG scheduling tự động
- Enterprise SLA, RBAC, lineage
- **Best for:** ML-heavy product teams, latency SLA nghiêm ngặt

- On-prem + cloud, data governance
- Tích hợp Spark + Flink streaming
- Lineage, versioning, GDPR-ready
- **Best for:** regulated industries (ngân hàng, y tế)

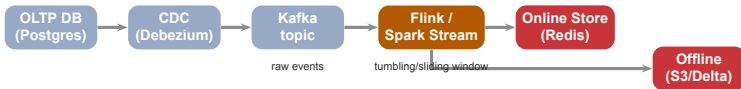
Lưu ý: Databricks Workspace Feature Store đã legacy (2024). Migrate sang Feature Engineering in Unity Catalog — không tạo mới project với Workspace FS.

Feature tính **khác nhau** giữa training và serving dẫn đến model hoạt động kém khi deploy mà không có lỗi rõ ràng.

- **Training:** `pd.DataFrame.mean()` trên toàn bộ dữ liệu lịch sử
- **Serving:** `running_mean(last_N)` chỉ N bản ghi gần nhất
- ⇒ Giá trị khác nhau, model sai lệch

- **Training:** parse datetime UTC
- **Serving:** parse local timezone
- ⇒ Feature “giờ trong ngày” lệch 7h (VN)

Lưu ý: Feature Store giải quyết skew: **một định nghĩa duy nhất** dùng cho cả training lẫn inference — không code riêng.



CDC: mọi UPDATE row → Kafka event

Aggregate: txn velocity 5min / 1h window

Feast Push API: app push trực tiếp event → online store

Có: fraud detection, dynamic pricing, real-time rec-sys

Không cần: churn prediction (daily batch OK), credit scoring

- **Latency:** <10 ms (per transaction)
- **Features:** velocity (txn/min), device fingerprint, amount-vs-7d-avg
- **Freshness:** streaming (Kafka → Redis online store)
- **Tooling:** Tecton + Redis cluster; Feast + Aerospike

- **Latency:** <50 ms (homepage personalization)
- **Features:** user embedding, last-N interactions, item popularity
- **Freshness:** hybrid (daily user-batch + stream last click)
- **Tooling:** Tecton (DoorDash, Atlassian); Hopsworks (regulated)

- **Latency:** <100 ms (price at search)
- **Features:** demand surge, supply, competitor price, time-of-day
- **Freshness:** sub-second (streaming demand signal)
- **Tooling:** Tecton end-to-end DAG, managed transformations



Feature Store thắng ở **3 dimensions**: feature reuse + train/serve consistency + low-latency lookup — không tool nào khác giải quyết cả 3.

- **Embedding cache:** Redis TTL 24h
- Hit rate 60–80%, giảm embedding cost
- **Batch embedding:** 1000 texts/request
- Throughput 50×, cost giảm 90%

- Track relevance score distribution theo tuần
- Nếu avg similarity giảm: điều tra embedding drift, data quality, query distribution
- P99 latency + index size growth

- Namespace isolation (Pinecone)
- Collection per tenant (Qdrant)
- Security + billing separation

- Create V1 → ingest → serve
- Update: build V2 offline → validate quality
- Blue-green switch → retire V1
- Rollback ngay nếu quality giảm

- **Recall@k**: tỷ lệ relevant docs trong top-k
- **MRR**: vị trí trung bình của kết quả đúng đầu tiên
- **nDCG@k**: xếp hạng chất lượng có weight
- Build 200-query golden set — regression-test mỗi lần đổi embedding hoặc chunk strategy

- P99 search latency + embedding latency
- Embedding cache hit rate (target >60%)
- Query distribution drift (alert nếu avg similarity giảm)
- Index size growth rate (trigger re-balance khi >120%)

- **Context Relevance**: retrieved chunks có liên quan không?
 - **Answer Relevance**: câu trả lời có trả lời query không?
 - **Groundedness**: câu trả lời có dựa trên context không?
- ⇒ RAGAS: faithfulness, answer relevancy, context precision

- **Langfuse** (open-source, OTel): tracing + eval tích hợp
- **Phoenix/Arize**: OpenTelemetry-native, RAG tracing
- **LangSmith**: LangChain-native, prompt + retrieval debug
- Chọn 1 tool — instrument từ ngày đầu tiên

40%

GlassDollar
(Qdrant)
hạ tầng cost ↓

3×

GlassDollar
user en-
gagement

1.6M

Alzheimer KG
edges (Mem-
graph)

<10 ms

PayPal fraud
feature lookup

GlassDollar (Qdrant): NL search Siemens/Mahle, 40% cost ↓, 3× engagement.

Memgraph P3C: KG 1.6M edges, sub-ms multi-hop traversal cho Alzheimer + diabetes.

PayPal: Tecton+streaming, <10 ms/txn fraud lookup.

DoorDash: Tecton DAG, sub-second pricing cho 60M+ orders/tháng.



ROI đo được: 40% cost ↓ + 3× engagement (GlassDollar) + <10 ms feature lookup (PayPal).
Không phải hype.

ML Era (2015–2022)

- **Vector store role:** candidate generation cho recsys; retrieval cho face / image search
- **Embeddings:** task-specific (FaceNet, two-tower recsys, doc2vec); train per-app
- **Index tooling:** FAISS / Annoy / NMSLIB embedded trong app process
- **Feature store:** tabular features (counts, ratios, lags) cho XGBoost / DNN tabular
- **Freshness:** daily / hourly batch là mặc định; streaming chỉ cho fraud

LLM Era (2023–2026)

- **Vector store role:** RAG retrieval, agent memory, semantic cache
- **Embeddings:** foundation models (OpenAI, BGE, Voyage); một model nhiều use case
- **Index tooling:** managed multi-tenant (Pinecone, Zilliz Cloud, Vertex Vector)
- **Feature store:** tabular *plus* embedding views; on-demand transforms cho prompt context
- **Freshness:** sub-second streaming là baseline cho fraud, pricing, real-time recsys

1. **Ingest 10,000 documents** Vietnamese vào Weaviate với bge-m3 embeddings (multilingual, tiếng Việt prod 2026)
2. **Semantic search:** “tìm tài liệu về cloud computing” → top 5 results với similarity scores
3. **So sánh:** keyword search vs semantic vs hybrid — semantic thắng trên paraphrased queries
4. **Feature Store:** Feast materialize user features → online lookup trong Jupyter notebook

- **Embedding:** dense vector biểu diễn nghĩa
- **Chunk:** đoạn text trước khi embed
- **Index:** cấu trúc ANN (HNSW, IVF, DiskANN)
- **Recall@k:** % relevant docs trong top-k
- **Cosine sim:** angular similarity, default metric
- **Quantization:** fp32 → int8/binary
- **Hybrid search:** BM25 + Vector → RRF merge
- **Reranker:** cross-encoder rerank top-N
- **Re-index:** rebuild khi đổi embedding model

- **Feature view:** schema + entity + source
- **Online store:** Redis/Dynamo, <10ms KV
- **Offline store:** Parquet/Delta, full history
- **Materialize:** batch load offline → online
- **PIT join:** as-of timestamp, no leakage
- **Train-serve skew:** feature mismatch bug
- **Push API:** stream event → online store
- **Feature service:** bundled features cho 1 model
- **Registry:** Feast metadata catalog

Lưu ý: **Feast key terms:** *Entity* = primary key (user_id, item_id). *On-demand feature* = computed tại request-time (không pre-materialize). *Feature service* = nhóm feature views cho một model.

Những ý chính cần nhớ trước khi sang bài tiếp theo

- 1 Hybrid Search (BM25 + Vector + RRF, $k = 60$): **91% recall** vs 78% dense-only — production default 2026.
- 2 GraphRAG khi câu trả lời là **relationship** (multi-hop, cross-document), không phải đoạn text. Kiến trúc layered: vector $\rightarrow$ KG.
- 3 Feature Store thắng cùng lúc **3 dimensions**: feature reuse, train/serve consistency, low-latency online lookup — không tool nào khác giải quyết cả 3.

Ngày 20: Model Serving & Inference Optimization

“Model accuracy 95% nhưng latency 3 giây — user đợi không nổi. Quiz + Milestone 1.”

- Hoàn thành Lab 19: Vector & Feature Store
- Đọc trước: vLLM docs — PagedAttention paper
- Ôn tập Chương 4 cho Quiz N20

Hỏi & Đáp

Câu hỏi về Vector DB, Hybrid Search, GraphRAG, hay Feature Store?



Cảm ơn!

AICB-P2T2 · Ngày 19

Vector Store & Feature Store

`lms.vinuni.edu.vn` · Slide & template trên LMS